

Lab 2: Process, Thread, and some applied Math

In this lab, you will create processes and threads, apply them to the computing of the famous math constant π . In this lab, you **MUST** create a directory “lab2/” and put all your files there. You **MUST present** this lab (as a group of 2 students) **AND** you must submit on Canvas all your working directory as a unique compressed file “lab2.tar.gz” using “tar” and “gzip” in Linux.

1) Process and fork()

We start with a simple code to create a process from another process using the fork() function from <unistd.h>. Create a new file “process01.c” in your “lab2/” directory and type in:

```
1 #include <stdio.h> // for the printf() function
2 #include <sys/types.h> // for the pid_t datatype
3 #include <unistd.h> // for the fork() function
4 #include <stdlib.h> // for the exit() function
5
6 int main() {
7     pid_t pid; // create a new process id variable pid
8
9     printf("It works!\n");
10
11     pid=fork(); // try to create a new process
12     if (pid==-1) { // something went wrong
13         printf("Error creating a new process... BAD!\n");
14         exit(-1);
15     } else if (pid==0) { // the CHILD receive 0 as pid value
16         printf("CHILD\n");
17     } else { // the PARENT receives the pid of the child as value
18         printf("PARENT\n");
19     }
20
21     printf("The End.\n");
22     return 0;
23 }
24
```

Try this code (gcc process01.c && ./a.out in the terminal) and answer the questions:

- When you use the fork() function, how to detect an error?
- When you use the fork() function, how to detect if you are in the CHILD process?
- When you use the fork() function, how to detect if you are in the PARENT process?

2) Process and fork() with Process IDs

Copy your file process01.c to process02.c, cp process01.c process02.c, then, edit the file process02.c to get:

```
1 #include <stdio.h> // for the printf() function
2 #include <sys/types.h> // for the pid_t datatype
3 #include <unistd.h> // for the fork() function
4 #include <stdlib.h> // for the exit() function
5
6 int main() {
7     pid_t pid; // create a new process id variable pid
8
9     printf("It works!\n");
10
11     pid=fork(); // try to create a new process
12     if (pid==-1) { // something went wrong
13         printf("Error creating a new process... BAD!\n");
14         exit(-1);
15     } else if (pid==0) { // the CHILD receive 0 as pid value
16         printf("CHILD with pid=%d and getpid()=%d\n", pid, getpid());
17     } else { // the PARENT receives the pid of the child as value
18         printf("PARENT with pid=%d and getpid()=%d\n", pid, getpid());
19     }
20
21     printf("The End.\n");
22     return 0;
23 }
24
```

Try this code (gcc process02.c && ./a.out in the terminal) and answer the questions:

- a) On Linux, how can you get help about the fork() function from the terminal?
- b) What is the difference between pid and getpid()?
- c) When you run this code, you obtain twice the “The End.” Text, why?
- d) When you run the code multiple times, you do not get the same PIDs each time, why?

3) Let's do process and math

In exercises 3.1 and 3.2, we will use a math formula to compute an approximation of PI:

$$\frac{1}{1^2} + \frac{1}{2^2} + \frac{1}{3^2} + \frac{1}{4^2} + \dots = \frac{\pi^2}{6}$$

You need to produce the list of all integers from 1 to N (with N big enough), square this integer (to a double), take 1 over this double, and sum them all. Then, you will obtain an approximation of pi squared over six and you need one multiplication and one more operation to get a result not far away from PI. We will do 1 billion iterations (line 6 in the code)!

3.1) One process to compute PI

In this exercise, you MUST complete the code “math-process01.c”, so that it computes an approximation of PI:

```
1 // MATH IN ONE PROCESS ONLY!!!
2
3 #include <stdio.h>
4 #include <math.h> // for the pow() function
5
6 #define ITERATIONS 1000000000
7
8 int main() {
9     printf("It works!\n");
10
11     double sum=0.0;
12     // true approximation of PI:
13     double approxPI=3.141592653589793238462643383279;
14     double myPI=0.0;
15     // sum of terms 1/(k^2) for k=1,2,...,N
16     for (int i=?; i<ITERATIONS; i++) {
17         sum+=(double)(1.0/?);
18     }
19     printf("The final sum is: %.30lf\n", sum);
20
21     // compute my value for pi
22
23     myPI=?;
24     printf("My value for PI is: %.30lf\n", myPI);
25     printf("True approx for PI: %.30lf\n", approxPI);
26     return 0;
27 }
28
```

Line 16, you must start from a number i=?.

Line 17, you must divide by a certain power using the pow(x,y) function according to the formula.

Line 23, you must find a formula to get PI, knowing the value of sum...

To try this code, you must compile it with gcc and include the math library in the process; to be able to use the pow() function... Perhaps, there is a “-lsomething” to add to your gcc command to use the math library?

If your code is working, please run the time command on your executable file (twice) to know how long it takes, of course, the speed depends on your hardware and software, everyone should obtain different values, for me it was around 24 seconds and my approximation of PI was 8 digits right “3.1415926”, not bad:

```
os2024@os2024-virtualbox:~/Desktop/c/lab2$ time ./a.out
It works!
The final sum is: 1.644934057834575025225376521121
My value for PI is: 3.141592644982389881391782182618
True approx for PI: 3.141592653589793115997963468544

real    0m23,567s
user    0m23,536s
sys      0m0,019s
os2024@os2024-virtualbox:~/Desktop/c/lab2$ time ./a.out
It works!
The final sum is: 1.644934057834575025225376521121
My value for PI is: 3.141592644982389881391782182618
True approx for PI: 3.141592653589793115997963468544

real    0m24,566s
user    0m24,512s
sys      0m0,026s
os2024@os2024-virtualbox:~/Desktop/c/lab2$
```

3.2) Two processes to compute PI

Now that it is working for one process, let's add a file "math-process-02.c" that creates two processes with fork(). I suggest that the CHILD would compute the even numbers in the sum and the PARENT will compute the odd numbers in the sum. Complete the code and run it:

```
1 // MATH IN TWO PROCESSES IN PARALELL!!!
2 #include <stdio.h>
3 #include <stdlib.h>
4 #include <math.h> // for the pow() function
5 #include <sys/types.h> // for the pid_t type
6 #include <sys/wait.h> // for the wait() function
7 #include <unistd.h> // for the fork() function
8 #define ITERATIONS 1000000000
9
10 int main() {
11     printf("It works!\n");
12     double sumCHILD=0.0, sumPARENT=0.0, sumFINAL=0.0;
13     // true approximation of PI:
14     double approxPI=3.141592653589793238462643383279;
15     double myPI=0.0;
16     char strCHILD[30]; char *ptr; // to exchange values between the CHILD and the PARENT
17     FILE *fp=fopen("shared_value.txt", "w+"); // we will save to a file
18
19     pid_t pid=fork();
20     if (pid<0) {
21         printf("Error creating a process...\n"); return -1;
22     } else if (pid==0) { // CHILD
23         for (int i=2; i<ITERATIONS; i+=2) {
24             sumCHILD+=(double)(1.0/pow((double)i,(double)2.0));
25         }
26         printf("The final ODD sum is: %.30lf\n", sumCHILD);
27         fprintf(fp, "%.30lf", sumCHILD); // we save the value in a FILE on the hard drive
28     } else { // PARENT
29         for (int i=1; i<ITERATIONS; i+=1) {
30             sumPARENT+=(double)(1.0/pow((double)i,(double)2.0));
31         }
32         printf("The final EVEN sum is: %.30lf\n", sumPARENT);
33         wait(NULL);
34         rewind(fp); // rewind the FILE pointer to position 1
35         char buffer[30]; // creates a buffer
36         fread(buffer, 1, 30, fp); // reads the value in the FILE
37         fclose(fp); // closes the file
38         printf("sum CHILD: %s\n", buffer); // this is the value as a STRING
39         sumCHILD=strtod(buffer, &ptr); // this is the value as a DOUBLE
40         printf("sum CHILD: %.30lf\n", sumCHILD);
41         sumFINAL=sumPARENT+sumCHILD;
42         printf("The final sum is: %.30lf\n", sumFINAL);
43         myPI=2/sumFINAL;
44         printf("My value for PI is: %.30lf\n", myPI);
45         printf("True approx for PI: %.30lf\n", approxPI);
46     }
47     printf("The End!\n");
48     return 0;
49 }
```

In this code, the CHILD is computing the even numbers sum and the PARENT will compute the odd numbers sum. The PARENT will wait, using the wait(NULL) function for the CHILD to finish, and will then sum the two half-terms sums to get the final sum. BUT, we need to SHARE the value of the CHILD sumCHILD to the PARENT process, there are many solutions to do so but the easiest way is to create a FILE, the CHILD will compute its own sum, write it to a file. Then, the PARENT will wait for the CHILD termination and read the value from the FILE (as a string), convert it to double, and compute the final sum. It is a COMPLICATED code, I know, but you must complete only lines 23, 29 and 43. The other lines work!

Compile and run this code, gcc math-process-02.c -lsomething (for the pow() function). Then, please, run this code twice and you should obtain a code that is twice (approximately) as fast as the one process version. Why? Because the two processes are running in parallel on your VirtualBox and you normally declared two processors in your VirtualBox. So, the processes are running one on each processor and it divides the run time by two. For me, I run twice and obtained:

```
os2024@os2024-virtualbox:~/Desktop/c/lab2$ time ./a.out
It works!
The final EVEN sum is: 0.411233514458643756306344130280
The final ODD sum is: 1.233700545629388578561247413745
The End!
sum CHILD: 1.2337005456293885785612474137
sum CHILD: 1.233700545629388578561247413745
The final sum is: 1.644934060088032445889894006541
My value for PI is: 3.141592647134283655674380497658
True approx for PI: 3.141592653589793115997963468544
The End!

real    0m12,571s
user    0m21,868s
sys     0m2,898s
os2024@os2024-virtualbox:~/Desktop/c/lab2$ time ./a.out
It works!
The final EVEN sum is: 0.411233514458643756306344130280
The final ODD sum is: 1.233700545629388578561247413745
The End!
sum CHILD: 1.2337005456293885785612474137
sum CHILD: 1.233700545629388578561247413745
The final sum is: 1.644934060088032445889894006541
My value for PI is: 3.141592647134283655674380497658
True approx for PI: 3.141592653589793115997963468544
The End!

real    0m11,934s
user    0m18,950s
sys     0m4,528s
os2024@os2024-virtualbox:~/Desktop/c/lab2$
```

About 12 seconds for each result. I got the same approximation as 3.1415926. And the time has been divided by almost 2, better than the previous one process only version!

Congratulations, you wrote a C program to compute in parallel using two processes, the approximation of PI. Let's see if it is easier with threads...

4) Let's do threads and math

4.1) Two threads

We will now build a new code using threads in one process instead of creating two processes.

Using the examples given in the lecture about threads, create two functions, associate them with two threads, one is responsible for the computation of the even sum and the other one for the computation of the odd sum. Then, you make them wait for their completion and the process will sum the two values and give the final result. Please complete and run the code:

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <math.h>
4 #include <pthread.h>
5 #define ITERATIONS 1000000000
6 // shared variables between the threads!!!
7 double sumEven=0.0;
8 double sumOdd=0.0;
9 // function to compute the Even sum
10 void *computeSumEven(void *param) {
11     double localSum=0.0;
12     for (int i=1; i < ITERATIONS; i += 2) {
13         localSum += 1.0 / (pow((double)i,2));
14     }
15     sumEven=localSum;
16     printf("THREAD EVEN finished\n");
17     return NULL;
18 }
19 // function to compute the Odd sum
20 void *computeSumOdd(void *param) {
21     double localSum=0.0;
22     for (int i=2; i<ITERATIONS; i+=2) {
23         localSum+=1.0/(pow((double)i,2));
24     }
25     sumOdd=localSum;
26     printf("THREAD ODD finished\n");
27     return NULL;
28 }
29 int main() {
30     pthread_t tEven, tOdd; // we need two threads
31     printf("It works!\n");
32     double approxPI=3.141592653589793238462643383279;
33     double myPI=0.0;
34     pthread_create(&tEven, NULL, ?, NULL); // thread Even
35     pthread_create(&tOdd, NULL, ?, NULL); // thread Odd
36     printf("Computing in progress...\n");
37     pthread_join(?, NULL); // thread Even is finished
38     pthread_join(?, NULL); // thread Odd is finished
39     double sum=sumEven + sumOdd;
40     printf("SUM is: %.30lf\n", sum);
41     myPI=?;
42     printf("My value for PI is: %.30lf\n", myPI);
43     printf("True approx for PI: %.30lf\n", approxPI);
44     return 0;
45 }
46
```

Try to time the execution twice, for me:

```
os2024@os2024-virtualbox:~/Desktop/c/lab2$ time ./a.out
It works!
Computing in progress...
THREAD EVEN finished
THREAD ODD finished
SUM is: 1.644934060088032445889894006541
My value for PI is: 3.141592647134283655674380497658
True approx for PI: 3.141592653589793115997963468544

real    0m12,080s
user    0m21,449s
sys     0m2,420s
os2024@os2024-virtualbox:~/Desktop/c/lab2$ time ./a.out
It works!
Computing in progress...
THREAD ODD finished
THREAD EVEN finished
SUM is: 1.644934060088032445889894006541
My value for PI is: 3.141592647134283655674380497658
True approx for PI: 3.141592653589793115997963468544

real    0m11,580s
user    0m20,906s
sys     0m1,928s
os2024@os2024-virtualbox:~/Desktop/c/lab2$
```

With the two threads code, we can use threads, and use our global variables, so that they are shared between the threads of the process! It is easier to write the code as two different functions.

4.2) Four threads

Now, you will create your own version of the threads program but with 4 threads instead of two! To do so, copy your file “math-threads.c” to “math-threads4.c”. In this file, create 4 functions instead of 2, create 4 threads instead of 2, your computation should now go 4 by 4 ($i+=4$) instead of 2 by 2 ($i+=2$). Compile and run your code and answer the questions:

- You do not divide the computing time by 2, having 4 threads instead of 2, why?
- How can you use the tool htop (you must install it: `sudo apt install htop`) to check the running of the threads in your terminal?

This is the end. In this lab, you learned to program processes and threads in C with an application to math problems. Congratulations, grattis!